

Loreley: Parallel, Multi-Branch Evolution of Complete Code Repositories

Quality over wall-clock time and longer searches

Mohan Chen · 2026-09

2.78×

Champion / QD time for 48 jobs

Paired geometric mean

7/7

Blocks where QD leads

when its 48 jobs finish

6/7

QD blocks reaching

the +0.50% threshold

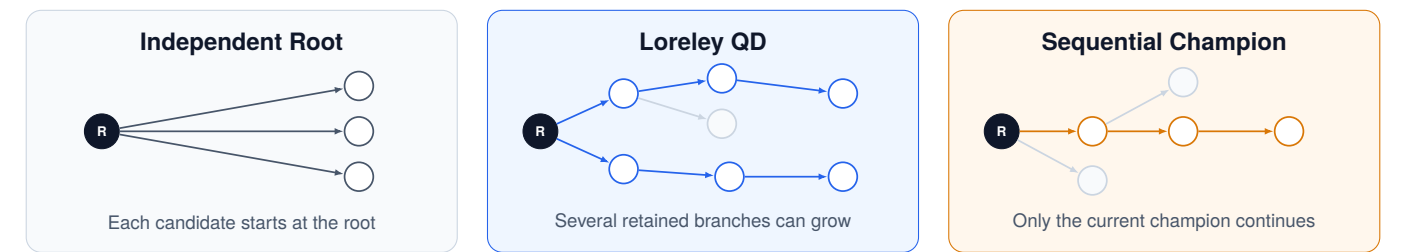
3–42

Jobs between a retained branch

and its final descendant

Main result Loreley’s quality-diversity (QD) search advances several branches in parallel and retains evaluated intermediate states as future parents. It reaches useful improvements sooner than Sequential Champion. At the same budget of 48 jobs, Sequential Champion has a slightly higher mean endpoint.

How the three strategies grow their lineages



Different search dynamics

Strategy	Parallel candidates	Cumulative edits	Median search time	Mean gain at 48 jobs	Threshold met at 48 jobs
Independent Root	Yes	No	3.66 h	+0.50%	2/7
Loreley QD	Yes	Yes	3.75 h	+0.82%	6/7
Sequential Champion	No	Yes	11.65 h	+0.96%	7/7

System workflow

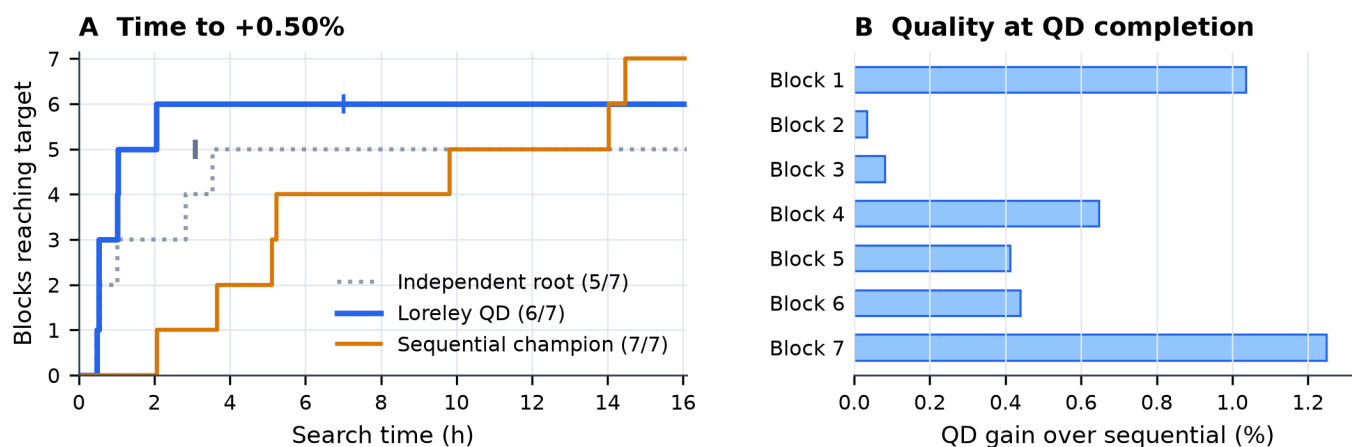
A coding agent starts from a buildable repository commit and makes cross-file edits in an isolated worktree. The project evaluator checks builds, tests, and edit scope, then records several performance metrics. Only commits that pass the checks enter the archive. Each Git commit preserves both code state and lineage. Archived commits can be selected again as parents or reference branches, so retaining a candidate changes the paths available to later jobs.

Controlled comparison

The Zstandard experiment has 7 paired blocks, with 48 candidate jobs per strategy in each block: 1,008 jobs in total. All strategies share the initial commit, agent routing, evaluator, and candidate-selection rule; only parent selection and scheduling differ. At jobs 8, 16, 24, 32, 40, and 48, validation selects a candidate for holdout measurement. Equal job counts measure search efficiency; equal wall-clock time measures delivery speed.

01 / WALL-CLOCK TIME

Useful candidates sooner

**Time to +0.50%**

The experiment uses a +0.50% compression-throughput gain as the threshold for a useful improvement. First attainment is read from each block's recorded checkpoints:

Strategy	Blocks	Median time
Loreley QD	6/7	0.78 h
Sequential Champion	7/7	5.23 h

QD is faster in all 6 blocks where both strategies reach the threshold. The paired geometric mean of Champion / QD attainment time is $6.76\times$. Times range from 0.48–2.05 h for QD and 2.08–14.46 h for Sequential Champion.

The critical path explains the time gap

QD runs 4 candidates concurrently, with median effective parallelism of 3.37. Sequential Champion runs one at a time, with median effective parallelism of 0.99. Total queue time per strategy and block is about 3–7 minutes, far below the hours separating them. QD advances several branches at once; Sequential Champion follows a single dependency chain.

Equal jobs and equal time**At 48 jobs, Sequential Champion finishes higher.**

Mean holdout gains are +0.96% for Sequential Champion, +0.82% for QD, and +0.50% for Independent Root. Sequential Champion has the highest mean endpoint at this fixed job count.

When QD finishes, it leads in all 7 blocks.

When QD completes 48 jobs, Sequential Champion's latest recorded checkpoint is job 8 in 4 blocks, job 16 in 2 blocks, and job 24 in 1 block. QD scores higher in all 7, with a median paired advantage of +0.44%.

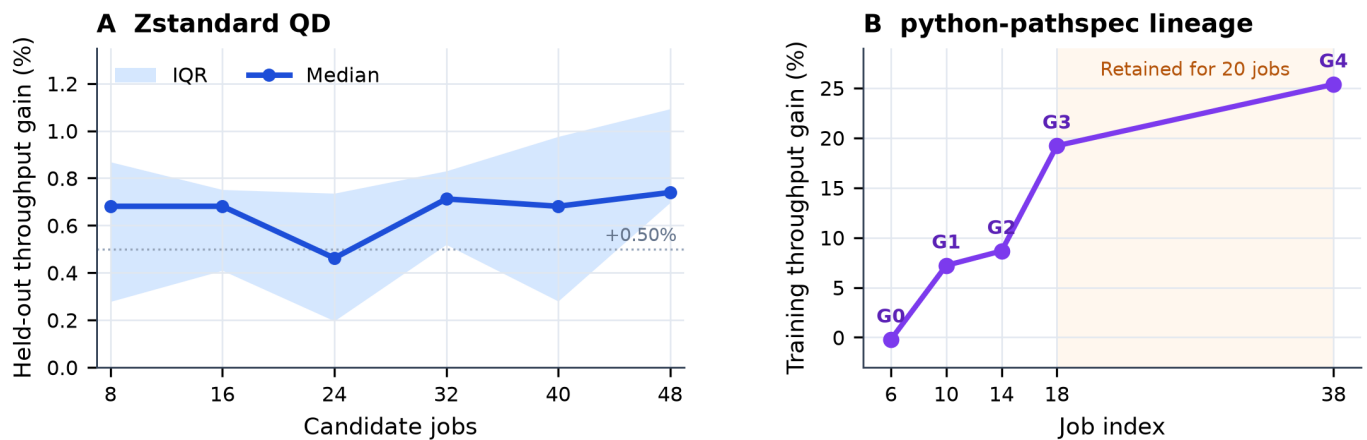
Under a time limit, QD delivers better candidates.

QD takes about as long as Independent Root while accumulating edits across generations. When delivery time matters, quality over wall-clock time is more informative than the endpoint after job 48 alone.

Wall-clock time is reconstructed from 1,008 candidate-job timestamps and covers candidate search only. At each checkpoint, validation selects a candidate before holdout compression throughput is measured. There is no interpolation between checkpoints. Vertical markers indicate that the threshold was not reached within 48 jobs.

02 / LONGER SEARCHES

New candidates in the second half

**20**Distinct primary parents per block
Median; range 15–23**5.5**Median job gap
before a parent is selected again**5–9**Maximum generation
reached across blocks**3–42**Jobs between a retained branch
and its final descendant**Gains continue from jobs 24 to 48**

Between jobs 24 and 48, the mean holdout gain of QD's selected candidates rises from +0.47% to +0.82%, and the median from +0.46% to +0.74%. Five of seven blocks improve; two remain unchanged. Three final candidates first appear at jobs 26, 38, and 43.

In the 220-job Zstandard run, every consecutive 16-job interval within the first 128 jobs adds a candidate to the archive. Jobs 129–220 add another 15 entries. The generation-4 candidate `fe39bee8` first appears at job 57 and achieves +1.17% on the existing holdout.

How the archive keeps search productive

In MAP-Elites, candidates compete within the same behavior cell. A state that is not the global leader can therefore remain available as a parent [1]. Loreley keeps a local Pareto front in each cell, preserving different trade-offs among compression, decompression, and worst-cell performance [2].

For two classes of combinatorial problems, Qian et al. prove that preserving intermediate states across cells lets MAP-Elites achieve polynomial expected runtime, while a globally competing evolutionary algorithm needs exponential expected time on some instances [3]. Loreley's lineage records show non-leading branches being selected again after several jobs and producing better descendants.

Paths spec lineage A branch is retained after reaching $1.19\times$ at job 18. The next 20 jobs start from other branches. At job 38, the retained branch is selected again and produces a generation-4 candidate at $1.25\times$: a +25.14% gain on the reference workload.

03 / APPLICATION

Wall-clock time and long-run gains

Results on three complete repositories

Target	Jobs	Final generation	Final candidate result	Search history
markdown-it-py	64	4	Independent validation: +6.75%	Edits accumulate at jobs 7/12/14/26
python-pathspec	64	4	Reference workload: +25.14%	Branch retained at job 18, continued at job 38
Zstandard (long run)	220	4	Existing holdout: +1.17%; fresh holdout: +0.89%	Candidate appears at job 57; 15 archive entries after job 128

What this changes in engineering decisions

1. Report time to threshold
When generation and evaluation can run in parallel, QD avoids putting all jobs on one champion chain. Here, QD reaches +0.50% sooner and leads Sequential Champion in all 7 blocks when its 48 jobs finish.

2. Track branch revisits
To evaluate an archive, record how often parents are selected, the gaps between branch revisits, when final candidates first appear, and gains between checkpoints.

3. Compare both jobs and time
At 48 jobs, Sequential Champion has the highest mean endpoint. At QD's completion time, QD leads in all 7 blocks. Report both comparisons.

When this approach fits

- Generating or evaluating one candidate takes minutes, and at least 3–4 candidates can run safely in parallel.
- Optimization requires compatible, cross-file edits over several generations.
- A project evaluator can check correctness, resource limits, and performance together.
- Complete Git states are needed for reproduction, rollback, and combining branches.

Summary

Loreley combines parallel candidate generation with a multi-branch archive that can be revisited. Parallel scheduling shortens the wall-clock time to a useful improvement. The archive keeps intermediate states that are not yet leaders, allowing later jobs to select them as parents and continue evolving them. In these experiments, QD reaches the improvement threshold sooner and delivers better candidates within the same wall-clock time. Longer runs keep adding candidates in their later stages, and retained branches produce better descendants after gaps spanning many jobs. Loreley's strengths are faster access to useful results and continued gains during longer searches.

Data sources. Zstandard comparison records, reconstructed quality–time measurements, and long-run records from three repositories. Verification scripts cover 126 checkpoint selections, endpoint statistics, and lineage counts.

References

[1] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015. URL <https://arxiv.org/abs/1504.04909>.

[2] T. Pierrot, G. Richard, K. Beguir, and A. Cully. Multi-objective quality diversity optimization. In J. E. Fieldsend and M. Wagner, editors, *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 139–147. ACM, 2022. doi: 10.1145/3512290.3528823. URL <https://doi.org/10.1145/3512290.3528823>.

[3] C. Qian, K. Xue, and R.-J. Wang. Quality-diversity algorithms can provably be helpful for optimization. In K. Larson, editor, *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 6994–7002. International Joint Conferences on Artificial Intelligence Organization, Aug. 2024. doi: 10.24963/ijcai.2024/773. URL <https://doi.org/10.24963/ijcai.2024/773>. Main Track.